International Journal of Forecasting 37 (2021) 1632–1653

Contents lists available at ScienceDirect

International Journal of Forecasting

journal homepage: www.elsevier.com/locate/ijforecast

Principles and algorithms for forecasting groups of time series: Locality and globality

Pablo Montero-Manso [a,*], Rob J. Hyndman [b]

[a] The University of Sydney Business School, Australia
[b] Monash Business School, Monash University, Melbourne, Australia

ARTICLE INFO

Keywords:
Time series
Forecasting
Generalization
Global
Local
Cross-learning
Pooled regression

ABSTRACT

Global methods that fit a single forecasting method to all time series in a set have recently shown surprising accuracy, even when forecasting large groups of heterogeneous time series. We provide the following contributions that help understand the potential and applicability of global methods and how they relate to traditional local methods that fit a separate forecasting method to each series:

- Global and local methods can produce the same forecasts without any assumptions about similarity of the series in the set.
- The complexity of local methods grows with the size of the set while it remains constant for global methods. This result supports the recent evidence and provides principles for the design of new algorithms.
- In an extensive empirical study, we show that purposely naïve algorithms derived from these principles show outstanding accuracy. In particular, global linear models provide competitive accuracy with far fewer parameters than the simplest of local methods.

© 2021 International Institute of Forecasters. Published by Elsevier B.V. All rights reserved.

1. Introduction

"*Unus pro omnibus, omnes pro uno.*" — D'Artagnan

Consider the problem of having to forecast many time series as a group. We might need to forecast tourist arrivals at all our resorts for next season, the demand for all products we offer at our retailer every week, server loads within a data center, the number of completed ride shares for all zones within a city on an hourly basis, and so on. The safest way (making the fewest assumptions) to approach this problem is to assume each of the time series in the set potentially comes from a different data generating process so it should be modeled individually, as a separate regression problem of finding a function to predict future values from the observed part of the series. This is the standard univariate time series forecasting problem; we call it the *local* approach to the problem.

From a statistical/machine learning perspective, the local approach suffers from one shortcoming: sample size. Temporal dependence and the short length of the series make time series forecasting a notoriously difficult problem. Individual time series cannot be modeled in a data-driven way because even basic models (e.g. linear models) will suffer from over-fitting. To overcome over-fitting, an excessive burden is placed on manually expressing prior knowledge of the phenomena at hand (assuming seasonal patterns, looking for external explanatory variables, calendar effects, separate long- and short-term effects, etc.) and restricting the fitting. This limits accuracy, since finding appropriate simple models is not always possible; it also limits scalability, since modeling each time series requires human supervision. As a result, ensembling simple forecast models has been the most widespread

* Corresponding author.
E-mail address: pablo.monteromanso@sydney.edu.au (P. Montero-Manso).

https://doi.org/10.1016/j.ijforecast.2021.03.004
0169-2070/© 2021 International Institute of Forecasters. Published by Elsevier B.V. All rights reserved.

approach to automatic time series forecasting (i.e. without prior information) for the last four decades.

A univariate alternative to the local approach, called the *global* (Salinas, Flunkert, Gasthaus, & Januschowski, 2020) or cross-learning (Smyl, 2020) approach, has been introduced to exploit the natural scenario where all series in the set are "similar", "analogous", or "related" (e.g. the demand for fiction books follows a similar pattern for all subgenres, stores, or cover designs). The idea behind the global approach is to introduce the strong assumption that all time series in the set come from the same process, because even when this assumption is not true, it will pay off in terms of forecasting accuracy. Global methods pool the data of all series together and fit a single univariate forecasting function (Rabanser, Januschowski, Flunkert, Salinas, & Gasthaus, 2020). The global approach overcomes the main limitation of the local approach: it prevents over-fitting because a larger sample size is used for learning compared to a local counterpart. On the other hand, the same function must be used to forecast all time series in the set, even when the series are different, which seems restrictive (and less general than local). The wider adoption of global methods has been prevented by the underlying assumption that they offer benefits over local methods *only* when time series in the set come from intuitively similar or related data generating processes.

However, recent empirical results show puzzlingly good performance of global methods applied over sets of time series that cannot be considered related (Laptev, Yosinski, Li, & Smyl, 2017). Notably, top results in the M4 competition (Makridakis, Spiliotis, & Assimakopoulos, 2020) use some form of globality, mostly in combination with local parts (Montero-Manso, Athanasopoulos, Hyndman, & Talagala, 2020; Smyl, 2020). Good performance with purely global methods (Gasthaus et al., 2019; Oreshkin, Carpov, Chapados, & Bengio, 2019) has soon followed these initial results. Because heterogeneous datasets are general (we can always add series from other sources to get a heterogeneous set), the implications of global methods working well for heterogeneous datasets can have a profound impact in forecasting practice. This can set a new state-of-the-art for automatic methods and open the possibilities of adopting data-driven techniques that are inapplicable for local methods, thus keeping up with the advances in machine learning.

Nevertheless, all cited pieces of empirical evidence are closely tied to the specific models used to achieve them, which differ among themselves, sometimes making contradictory claims about the causes of this performance. There is a lack of understanding of the underlying general principles behind this good performance (even if they exist). These principles can help solidify existing knowledge, accelerate technology transfer to real-world applications, and guide future improvements in the field.

Motivated by these possibilities and needs, in this paper we compare global and local forecasting algorithms from the perspective of statistics and machine learning applied to a finite number of series, each of finite length, and for a finite horizon of interest. This leads to the following contributions.

- In a formal setting of forecasting a set of time series, we show that local and global algorithms can produce the same forecasts, so they can approximate the series equally well (Section 2). A local algorithm might learn a different function to forecast each series in the set, but its forecasts can *always* be replicated by a single function learned by a global algorithm. Global algorithms in the general setting of supervised learning (i.e. pooling together regression problems with arbitrary input–output pairs) do not exhibit this property. This result can be understood as an existence result that motivates the use of global algorithms for a wider range of problems than previously thought, since we require no assumptions about the similarity of the series in the set.
- We derive generalization bounds of local and global algorithms for forecasting sets of time series (Section 3). These bounds relate model complexity (measured as the number of functions that can be fitted to data) and sample size to the difference between training (in-sample) error and testing (out-of-sample) error. The bounds are agnostic of specific models and the distribution of the data, allowing us to reason without knowing the specifics. We show that the complexity of local algorithms grows with the size of the set, so it can easily surpass the constant complexity of global algorithms. For example, for sets of moderate size, a local algorithm, such as fitting exponential smoothing models to each series in the set, is (surprisingly) more complex than a single global deep network with thousands of parameters. Therefore, the individually simple local method might have poorer generalization. These results are evidence that global models are a version of multi-task learning (Zhang, 2015) and have antecedents in sequence-to-sequence models for forecasting (Mariet & Kuznetsov, 2019).
- Derived from the complexity analysis, we provide principles for designing forecasting algorithms for sets of time series that lead to improved forecasting accuracy. These principles can serve as a "toolbox" which can be adapted to specific forecasting problems. Furthermore, these principles unify local and global approaches under the common framework of controlling model complexity.
- We motivate the use of global algorithms with large complexity (Section 3.4). We highlight three principles:

  Adding features : For example, manual features can be added, such as high-degree polynomials, and alternate model classes can be considered, such as nonparametric/kernel methods, deep networks, or regression trees. This in turn is strong theoretical support for the current empirical evidence based on deep learning.

  Increasing memory : Restricting the class of models to finite memory autoregressions, global models with increased complexity can be realized by increasing the memory/order of autoregression (Section 3.5). In addition to the

benefits of better fitting, this shows that long memory patterns might be easier to pick with global models.

Partitioning : We cast the problem of forecasting a set of time series as a spectrum between two extremes based on how the set is partitioned. Partitioning the set into single series leads to the local approach, while the global approach works on the trivial partition that keeps the set intact. We show that the complexity of a learning algorithm can be controlled by the granularity of the partitioning (Section 3.6). This result highlights the trade-off in sophisticated ways of partitioning, such as data-driven partitioning, leading to clustering and its fuzzy weighted alternatives.

- We showcase the strength of these theoretically derived principles in a large empirical study on benchmark datasets (Sections 4–5). We incorporate these principles in global forecasting methods based on existing classes such as linear models, high-order polynomials, and deep networks (MLP ReLu). To isolate the contribution of the principles, these methods are *naïve* in the sense that they do not rely on feature engineering, seasonality, advanced deep learning architectures, hyperparameter tuning, preprocessing steps, or model selection/combination. These methods achieve good accuracy through a wide range of benchmark datasets. We find compelling empirical evidence towards global methods in the performance of linear models: while theoretically the complexity of a global algorithm must be larger than the one a local algorithm assigns to each individual series, we find that in practice it does not need to be much larger. This means that global methods can be both simpler (three orders of magnitude fewer parameters) and more accurate.
- We provide a theoretical explanation for recent empirical results on the performance of global methods (Section 6). The main cause we propose—that global methods can have higher complexity and still generalize better—differs from the explanations given in related literature, which we show are too narrow. We continue with a critical analysis of related literature claims about clustering, memory, and preprocessing from the theoretical point of view of the elements involved in the bounds. Source code available at: https://github.com/pmontman/paper-global-forec-princip.

2. Equivalence of global and local approaches for forecasting a set of time series

"*Each forecast that can be expressed by a local algorithm can also be expressed by a global algorithm.*"

2.1. Local and global learning algorithms

We compare two approaches to forecasting a set of time series. The *local approach* fits a function to each time series in the set. The *global approach* fits the same function to all time series in the set. Both approaches are learning algorithms, functions that take data as input and produce forecasting functions as output. These forecasting functions, in turn, take the observed part of a time series as input and produce the future part (forecast) as output. Because the global approach is restricted to producing the same function for all time series in a set while the local can use a different function for each series, we might think the global is more limited than the local approach. It turns out that under realistic assumptions this is not true: local and global approaches do not differ in the forecasts they are able to produce.

Let $\mathbb{S}$ be the set of univariate time series of size K, $\mathbb{S} = \{X_i \in \mathbb{R}^T, i = 1 \dots K\}$, and we allow $\mathbb{S}$ to have repeated time series. X_i denotes the observed time series, vectors of real numbers, assumed to be of the same finite length T without loss of generality (e.g. one can carefully pad the shorter series). We are interested in the future of each time series until a *finite* number of time steps H; i.e. each time series X_i has a future part which is a vector in $\mathbb{R}^H$. Forecasting functions are functions from the observed time series to the future part:

$$f : \mathbb{R}^T \to \mathbb{R}^H.$$

Let A_L be a local learning algorithm: a function that takes a time series X_i and returns a forecasting function f_i. That is,

$$A_L : \mathbb{R}^T \to (\mathbb{R}^T \to \mathbb{R}^H).$$

The same local algorithm is applied to each time series in $\mathbb{S}$, producing a different function for each X_i, and as an algorithm it must produce the same output given the same input.

A global method A_G is also a learning algorithm, but it only produces a single forecasting function, g, for the whole $\mathbb{S}$,

$$A_G : \mathbb{S} \to (\mathbb{R}^T \to \mathbb{R}^H)$$

Proposition 1 (*Equivalence of Local and Global Algorithms for Time Series Forecasting*)**.** *Let*

- $\mathbb{A}_L$ = *set of all local learning algorithms;*
- $\mathbb{A}_G$ = *set of all global learning algorithms;*

Then

1. *For any A_G in $\mathbb{A}_G$, there exists an A_L in $\mathbb{A}_L$ so that $(A_G(\mathbb{S}))(X_i) = A_L(X_i)$ for all X_i in $\mathbb{S}$*
2. *For any A_L in $\mathbb{A}_L$, there exists an A_G in $\mathbb{A}_G$ so that $A_L(X_i) = (A_G(\mathbb{S}))(X_i)$ for all X_i in $\mathbb{S}$*

To prove Proposition 1 we study the two implications separately.

1. Given the function $g = A_G(\mathbb{S})$, an A_L that sets every $f_i = g$ independent of the series would produce the same forecasts.
2. In this scenario, we fix the output of the local algorithm, $\{f_i, i = 1, \dots, K\}$, and show that there is a g that can approximate them. We consider two possibilities:

(a) $X_i \neq X_j$ for all $i \neq j$.
This means every X_i is unique in $\mathbb{S}$. The set of all $f_i(X_i)$ is finite since $\mathbb{S}$ is finite, with cardinality less than or equal to K. Then g must be a function that maps from a finite set to another of less or equal size, and such a function exists. The codomain of g is equal or smaller than its domain, so there is a function that hits every element of the codomain at least once (the pigeonhole principle). We can even explicitly construct g if we are given all the f_i, by our favorite universal approximator: polynomials, kernels, etc.
(b) $X_i = X_j$ for some $i \neq j$ but $f_i(X_i) \neq f_j(X_j)$
In this case there is no global function g because it would need to produce two different values for the same input, which goes against the definition of a function. This is why global methods are more restrictive than local methods for regression problems in general. In the context of supervised learning, regression deals with finding a function mapping from X_i to the arbitrary "targets" y_i, with both X_i and y_i being the inputs to the algorithm. Therefore it can happen that $X_i = X_j$ but $y_i \neq y_j$. A local method would be able to find such f_i and f_j so that $f_i(X_i) = y_i$ and $f_j(X_j) = y_j$, but there is no single function g that can produce such a mapping. In time series forecasting, this situation cannot happen because there is no arbitrary mapping to be found. The only inputs to the algorithm are the X_i. Because a learning algorithm is also a function, equal input produces equal output. For a local algorithm A_L we have $X_i = X_j \Rightarrow A_L(X_i) = f_i = A_L(X_j) = f_j$, so situation (b) is not possible. □

2.2. Finite memory

Proposition 1 refers to forecasting functions that take the whole series into account, i.e. its T observed time periods. It is often the case that interest is restricted to forecasting functions using only the most recent M observations of the series, $M < T$. These are called finite memory models, autoregressive models, or Markov models. In some contexts, M is called the order of the autoregression, the size of the memory, or the receptive field. Finite memory models are useful because they deliver a powerful simplification of the world, and because they are easier to fit to data and to analyze. Many natural phenomena can be explained by these models (the ubiquitous ARIMA class is an example of finite memory).

In the setting of Proposition 1, there is no guarantee of the equivalence of local and global approaches if both produce finite memory functions. A simple example is of two time series having the same M recent observations, but differing at some other point in the T observed time points. A local approach might produce different functions for these two series since they are different, so their forecast could be different, but any global method will produce the same forecasts for these two series. This is interesting because, to guarantee equivalence, the global approach must use longer memory than the local approach. This is a key concept that leads to a guiding principle in the design of global methods: longer memory to approximate the expressive power of local methods.

3. Generalization relationship between local and global approaches

The main goal of forecasting is to produce accurate predictions on unseen data according to some error measure. We have seen in the previous Section 2 that local and global algorithms can produce the same forecasts, and therefore potentially achieve the same accuracy, but Proposition 1 does not tell us about how two fixed algorithms compare in practice.

3.1. Generalization error bounds

We study how well local and global algorithms generalize. The goal of studying generalization is to determine about how far the expected error on unseen (out-of-sample) data can be from the error an algorithm makes in the observed (in-sample) data. The difference between out-of-sample error and in-sample error is called the *generalization error* or *generalization gap*.

Low generalization error does not imply low out-of-sample error, but only that it will be close to the in-sample error. High generalization error is usually related to over-fitting. Theoretical analysis of generalization produces results in the form of probabilistic bounds for the generalization error; it gives us the probability that the out-of-sample error lies within a desired distance of the in-sample error. The bound can also be interpreted as a confidence interval for the expected out-of-sample error.

We use a basic result in machine learning about generalization error (Abu-Mostafa, Magdon-Ismail, & Lin, 2012) that relates how far the out-of-sample error, E_{out}, can be from the in-sample error, E_{in}, in terms of an i.i.d. dataset of size N and a model $\mathcal{H}$ (also termed the hypothesis class). The model $\mathcal{H}$ is a finite set of functions that can be potentially fitted to the data. The size of $\mathcal{H}$, denoted by $|\mathcal{H}|$, is the *complexity* of $\mathcal{H}$. The basic bound has the form

$$E_{out} < E_{in} + \sqrt{\frac{\log(|\mathcal{H}|) + \log(\frac{2}{\delta})}{2N}},$$

with probability of at least $1 - \delta$.

To simplify notation, we are assuming E_{in} to be the in-sample average and E_{out} to be its expectation, both calculated over loss values of the predictions of one forecasting function. This function is chosen from $\mathcal{H}$, picked after exploring all alternatives (e.g. the one that minimizes E_{in}). We can think of a learning algorithm as the way of picking this function from $\mathcal{H}$, though in practice a learning algorithm is tied to a specific $\mathcal{H}$. We assume that the error measure, also named the loss function, takes values in [0, 1].

To apply this bound we *only* need information about the sample size, maximum and minimum values for the error, and the number of functions that we consider in our model. It is a good compromise of generality: introducing additional assumptions makes the scope too narrow, and making it more general leads to vacuous statements.

This bound assumes the worst possible situation with the information we have. It is valid for any distribution of errors taking values in [0, 1]. In fact the worst distribution

is either the min or max error with half probability each. The model $\mathcal{H}$ can be any set of functions and the algorithm can pick from it in any way possible, for which the worst case is testing the error of all functions $\mathcal{H}$ before picking one.

The assumption of bounded loss seems limiting but is of great general applicability for what we get from it. The range [0, 1] is arbitrary for notational convenience, when the natural range of the errors is different (e.g. in our experiments), one can apply a scaling factor. We highlight three situations where, after rescaling, the bound can be applied naturally:

- The measurement units have maximum and minimum values. E.g. the number of patients arriving at a hospital has a clear lower bound and an upper bound can be reasonably defined.
- The error measure is relative. E.g. sMAPE is between 0 and 200 even when we do not know the bounds of the variable.
- We are interested in errors over a certain threshold, in which case we can assume the error is bounded. The bounds assume the worst-case scenario where maximum and minimum errors each have a 50% chance of happening, so it is a safe worst-case scenario (depending on our needs).

More sophisticated notions of complexity other than the size of $\mathcal{H}$ exist, which can make the bound tighter. These require additional information about $\mathcal{H}$ or the way of choosing a function from $\mathcal{H}$. For example, the relation between the number of steps in stochastic gradient descent and the generalization error (Hardt, Recht, & Singer, 2016) could be adapted to get equivalent analyses, provided we know certain properties of our models. In the context of time series, Mariet and Kuznetsov (2019) analyze the sequence-to-sequence models for time series forecasting using the concepts of Rademacher complexity and discrepancy, a notion of the nonstationarity series that includes model and loss function.

The insights derived from our results do not depend on an accurate notion of complexity, since they come from examining *relative* performance. This bound is particularly useful for reasoning at a general level. We do not need information about the distribution of the data (just the min and max error values) and about the hypothesis class (as long as we know its finite size).

To use this bound in the context of a time series, where observations are not i.i.d., we need to make the extra assumption of N being an *effective* sample size, which somehow grows with the length of the series T, but is less than T in general. This is a feasibility requirement; if we cannot guarantee that the longer we observe a time series, the better we can approximate it, then it is not possible to learn from time series data. See Theorem 1 in Kuznetsov and Mohri (2016) for a discussion on the notion of effective sample size in non-i.i.d. processes; see also (McDonald, Shalizi, & Schervish, 2017) (Theorem 4) and Kuznetsov and Mohri (2017). In our case, N "encapsulates" or "abstracts away" important details about the underlying processes (degree of nonstationarity, dependence structure, etc.) which must be assumed otherwise. Arguably, we use automatic time series forecasting *precisely* because we do not know these details. Moreover, because we are focusing on the relative performance of local and global approaches, it is not important to know the exact effective sample size, as N will be the same for both.

3.2. Comparing local and global methods on a single time series

We can directly apply this bound to compare local and global approaches in the context of the out-of-sample error of one isolated time series in the set. We adopt the common practice of considering that one approach is better than the other if its bound is better. This is a heuristic: better bounds do not guarantee better out-of-sample error. We can view this heuristic as a form of penalization for model selection, similar to Akaike's Information Criterion, though we do not use it as such.

We can show a result that confirms our intuition and the main rationale behind the preference for local methods:

> *"For a time series in isolation and for local and global algorithms with the same hypothesis class, the local approach has a better worst-case out-of-sample error than the global approach".*

This occurs because the best local in-sample error will be at most the same as the best global in-sample loss. The global approach must pick the function from $\mathcal{H}$ that minimizes the in-sample loss of all time series in the set, while the local approach only focuses on the current time series of interest. The remaining terms in the bound are equal for both approaches, so it follows that the bound is better for the local. One could argue that the hypothesis class of the global approach is effectively smaller because it needs to fit to other data. Nothing can be said in this regard at this level of generality, since we are making no assumptions on the distributional relationship *between* series in the set, not even independence. Without the independent series assumption, the set could have been chosen to be adversarial for global methods. The assumption of independence is necessary in the mainstream machine learning literature in order to make claims about generalization (e.g. we are already assuming "approximately" independent observations within a series), so it follows that it should be introduced at the between-series level.

3.3. Comparing local and global methods on groups of independent time series

When we introduce the assumption that the series in $\mathbb{S}$ are independent and focus on the average error across all time series in the set, the comparison changes. The assumption of independence between series is not strong, in the sense that dependent time series can be analyzed from a multivariate point of view, e.g. with vector autoregression, rather than with a global model. On the other hand, averaging errors across different time series might

be pointless in some scenarios, such as when time series have different scales or are measuring different units.

We name E_{in}^{Local} the average in-sample error across all time series of the local approach, and E_{out}^{Local} its out-of-sample expectation. The equivalent terms for the global approach are E_{in}^{Global} and E_{out}^{Global}. The local approach uses hypothesis class $\mathcal{H}_i$ for time series X_i, and the global approach uses only one hypothesis class, $\mathcal{J}$, for the whole set of series.

This scenario results in the following bounds for local and global approaches.

Proposition 2 (*Generalization Bounds for Local and Global Algorithms of Finite Hypothesis Class*)**.** *Local and global approaches have the following generalization bounds:*

$$E_{out}^{Local} < E_{in}^{Local} + \sqrt{\frac{\log\left(\prod_{i=1}^{K} |\mathcal{H}_i|\right) + \log(\frac{2}{\delta})}{2NK}}$$

$$E_{out}^{Global} < E_{in}^{Global} + \sqrt{\frac{\log(|\mathcal{J}|) + \log(\frac{2}{\delta})}{2NK}},$$

with probability of at least $1 - \delta$.

Assuming independence between the time series in $\mathbb{S}$, and given an effective sample size N in each time series, we can follow the same reasoning that results in the original classical bound. This follows from the Hoeffding inequality (Hoeffding, 1963), restated here as:

$$P(\bar{Y} - \mathbb{E}(\bar{Y}) \geq t) \leq 2e^{-2Nt^2},$$

for independent random variables $Y_1, \ldots, Y_N$, where Y_i might come from any probability distribution with bounded support in $[0, 1]$, $\bar{Y} = N^{-1} \sum Y_i$, and $t > 0$. We can apply the Hoeffding inequality by considering that Y_i is *equivalent* to the in-sample errors made by one a priori fixed function in a time series of effective sample size N. Then $\bar{Y}$ will be the average in-sample error and its expectation $\mathbb{E}(\bar{Y})$ the expected out-of-sample error.

Instead of fixing the hypothesis, if we pick one hypothesis over all possible hypotheses in a set $\mathcal{H}$, the bound relaxes. We can apply the union bound (Boole's inequality) to the error when picking between many functions, such that:

$$P(\bar{Y} - \mathbb{E}(\bar{Y}) \geq t) \leq |\mathcal{H}| 2e^{-2Nt^2}.$$

This last step is similar to the Bonferroni correction in multiple hypothesis testing.

When we move from one time series to averaging over K independent time series in the set, we get NK samples, assuming for simplicity that all series have the same effective sample size.

For the local approach, the size of the hypothesis class is the size of the Cartesian product of all the local hypotheses $\mathcal{H}_i$ used to fit the series, while for the global approach, the size is $|\mathcal{J}|$. For example, if the local approach picks between three functions for each series ($|\mathcal{H}_i| = 3$), it is equivalent to picking between 3^K functions for the full set of K series.

For the local approach:

$$P(E_{out}^{Local} - E_{in}^{Local} \geq t) \leq \prod_{i=1}^{K} |\mathcal{H}_i| 2e^{-2KNt^2}.$$

To get to Proposition 2, we rewrite the right-hand side of the previous inequality as our tolerance level δ:

$$\prod_{i=1}^{K} |\mathcal{H}_i| 2e^{-2KNt^2} = \delta$$

$$\Rightarrow \qquad t = \sqrt{\frac{\log\left(\prod_{i=1}^{K} |\mathcal{H}_i|\right) + \log(\frac{2}{\delta})}{2NK}}.$$

So

$$P\left(E_{out}^{Local} - E_{in}^{Local} \geq \sqrt{\frac{\log\left(\prod_{i=1}^{K} |\mathcal{H}_i|\right) + \log(\frac{2}{\delta})}{2NK}}\right) \leq \delta.$$

The complementary event corresponds to the result in Proposition 2. The reasoning for the global approach is analogous.

Key to this result is the fact that the Hoeffding bound does not require samples to be identically distributed, so the series in $\mathbb{S}$ may vary wildly. They can have *any* distribution as long as they are independent (and the error is bounded between 0 and 1). The bound is applicable to *independent copies* of $\mathbb{S}$. To apply it to *new* observations from the same series in $\mathbb{S}$, these should come in the same proportion as the effective sample sizes.

The bounds derived in Proposition 2 allow us to compare how local and global algorithms generalize in a (probabilistic) worst-case sense before we even see the data and hypothesis classes. We can predict how altering the elements involved in these bounds will affect generalization, and we can propose algorithms inspired by these predictions.

3.4. Relative complexity of the hypothesis classes of local and global approaches

Imagine we are given a local algorithm and we want to find a global algorithm that has the same or better performance in a given dataset. Under the paradigm (or heuristic) of "better bounds mean a better algorithm", we can try to find a global algorithm that produces the same in-sample error as the local reference. Then, from Proposition 2, it follows that the relative performance is totally determined by the respective complexity terms. Proposition 1 becomes relevant now because it shows that a global algorithm exists that can match the errors of a fixed local approach. In practice, we can think of having a "budget of complexity" to reach the in-sample loss equivalent to the local approach. If we get to it within the budget, we get better generalization than the local alternative and therefore better performance. We aim for equal in-sample loss to isolate the effect of the complexity, which is what we can control before seeing the data.

Assuming $E_{in}^{Local} = E_{in}^{Global}$ and $E_{out}^{Local} = E_{out}^{Global}$, we can match the two bounds in Proposition 2 to get:

$$\prod_{i=1}^{K} |\mathcal{H}_i| = |\mathcal{J}|,$$

with the = symbol *overloaded* to mean they have the same worst-case guarantees. We can read this result as saying

that the complexity of the local approach grows with the size of the set, while the complexity of the global approach remains constant. Assuming a state-of-the-art local algorithm as the baseline (such as an ARIMA fitted to each series), a global approach can be at least as complex as the sum of all individual ARIMAs in the set.

This leads us to prescribe the use of comparatively more complex hypothesis classes for global algorithms. The common ways of increasing complexity are by adding features/variables (such as nonlinear transformations) or changing the model family altogether (e.g. from a linear to a neural network). The experiment in Section 5.2 shows empirical evidence of this principle. This result can serve as an explanation for the recent success of global models, which are generally more complex that the traditional local models. The relationship between global and local models is qualitatively the same as multi-task and single-task learning frameworks (Zhang, 2015), as well as sequence-to-sequence models in time series (Mariet & Kuznetsov, 2019). The important difference coming from Proposition 1 is that global models can approximate as well as local models, such that they can be compared in a general setting. We can consider global models as a specific class within the larger framework of multi-task learning, and potentially some of the results in the latter could be transferred to the former.

3.5. Relative memory of local and global autoregressive models

In our analysis of finite memory models (Section 2.2) we saw that global autoregressive methods must have larger memory than local methods to be able to express the same forecasts. Increasing the memory of an autoregressive model can be analyzed from Proposition 2: it increases the complexity of the hypothesis class. It can be seen as a form of featurization more specific to time series than other forms of complexity control.

We can use a reasoning analogous to that in Section 3.4 to isolate how memory affects generalization. Assume that the local and global approaches use autoregressive models (which can be nonlinear), with one parameter per order of autoregression (or size of the memory). Then the complexity of the hypothesis class is completely determined by the order of the autoregression. Even though in theory there are infinite autoregressive models that can be fitted to data (and thus the bounds in Proposition 2 become infinitely loose), we can restrict ourselves to the realistic scenario of using double floating-point numbers for the parameters. This limits the number of possible models that can be fitted, since there are roughly 2^{64} possible values for each parameter. If we consider the memory of the local model fitted to each series X_i to be L_i and the memory of the global approach G, we can explicitly compare the complexities for the local and global algorithms. Substituting in the result of Section 3.4,

$$\prod_{i=1}^{K} 2^{64L_i} = 2^{64G},$$

and then taking logarithms and simplifying, we obtain

$$\sum_{i=1}^{K} L_i = G.$$

> *"Local and global autoregressive algorithms with the same performance have the same total memory".*

As in Section 3.4, it follows that the global approach can have much larger memory than that used by a local approach for each time series. While other complexity-controlling procedures, such as adding nonlinearities, can be borrowed directly from the common machine learning literature, this result is more specific to time series. Proposition 1 indicates that global models need longer memory and we show now that can they afford much more memory than local alternatives. This leads us to prescribe the use of global algorithms with relatively larger memories. Empirical results of this prescription can be found in Section 5, particularly in Section 5.1, highlighting the importance of controlling this parameter.

3.6. Relative complexity of partitioning $\mathbb{S}$

From a higher level of abstraction, we can see local and global as special cases of an algorithm that acts on a *partition of* $\mathbb{S}$, instead of on $\mathbb{S}$. This algorithm pools together the series in each element of the partition of $\mathbb{S}$, fits a single model within each partition, and forecasts each series in that subset using that function. Therefore, the "local version" of this algorithm works on the *atomic* partition ($\{$all subsets of $\mathbb{S}$ of size 1$\}$) and the "global version" works on the *trivial* partition ($\{\mathbb{S}\}$).

The bounds in Proposition 2 can be adapted to this new level of generality. Introducing the cardinality of the partition as P, the bounds become:

$$E_{out}^{Part} < E_{in}^{Part} + \sqrt{\frac{\log(\prod_{i=1}^{P} |\mathcal{H}_i|) + \log(\frac{2}{\delta})}{2NK}},$$

with probability of at least $1-\delta$. As in previous bounds, to simplify notation we are assuming that all P elements of the partition contain the same number of series. For the trivial partition $P = 1$ we recover the global algorithm, and for the atomic partition $P = K$ we recover the local algorithm.

Considering all other possible partitions of $\mathbb{S}$ leads to another way of controlling complexity. A global algorithm may increase its complexity by partitioning, in addition to exploring alternate hypothesis classes or increasing its memory. Similarly, local algorithms can reduce their complexity by pooling together time series in small groups.

Partitioning a set is also specific to time series forecasting, similar to the result in Proposition 1, because we are not interested in forecasting new time series, just the unobserved part of the given set.

These bounds refer to the complexity for a specific partition of $\mathbb{S}$ fixed before seeing the data, in the same sense that the Hoeffding inequality considers a single hypothesis set beforehand. Choosing between different partitions

based on data is commonly called clustering and leads to an increase in complexity. While clustering can be analyzed with Hoeffding-based bounds, the worst-case analysis (agnostic of the clustering algorithm) leads to vacuous bounds, due to the combinatorial explosion of the number of partitions in a set. Partitioning is also related to multi-task learning (as a form of soft partitioning), as discussed in Section 6.3.

We leave clustering outside the scope of the paper. Instead, to showcase the practical usefulness of the principle of partitioning, we propose the most naïve form of partitioning: random partitioning into subsets of equal size. Even this simple idea can be effective when the global model cannot reach good in-sample loss levels due to the size of the set. Section 5.3 shows empirical evidence of this prescription.

4. Experimental setup

We describe in this section the common elements of the experiments, such as the local and global algorithms we compare, the datasets, and the error metrics.

4.1. Local benchmark methods

We consider some methods implemented in the `forecast` R package (Hyndman, Athanasopoulos et al., 2020; Hyndman & Khandakar, 2008) for the benchmark local methods.

- **auto.arima**: Seasonal ARIMA with model selection by AICc.
- **ets**: Variants of Exponential Smoothing selected by AICc.
- **theta**: Implementation of the theta method, equivalent to simple exponential smoothing with drift.
- **TBATS**: Automatic multi-seasonal state space model with ARMA errors.
- **STL-AR**: STL decomposition with AR errors.

The first three (auto.arima, ets, and theta) are general-purpose methods particularly well suited for monthly, quarterly, and annual data. TBATS and STL can also handle multiple seasonalities that arise in daily and weekly data. All these methods require the frequency of the series to be pre-specified, unlike the global methods we consider.

We consider these methods state-of-the-art based on the results of the M4 competition (Makridakis et al., 2020). The top entries in that competition achieved their accuracy by ensembling these methods (Jaganathan & Prakash, 2020; Montero-Manso et al., 2020; Pawlikowski & Chorowska, 2020) or by combining them with neural networks (Smyl, 2020).

We do not consider ensembling as a benchmark local method, since ensembling is a technique that can also be applied to global methods. The more accurate the individual models that enter the ensemble, the more accurate the results of the ensemble, so the target accuracy is that of the individual methods.

4.2. Global methods

All global methods are based on autoregressive or finite memory models. Each series is lag-embedded into a matrix at the given AR order, and then these matrices are stacked together to create one big matrix, achieving data pooling. The different model classes minimize a loss on this large global matrix, as in a standard regression problem, with the final column of the matrix representing the target values or dependent variable, and the remaining columns representing the input or predictor variables. All global models produce one-step-ahead forecasts. Longer forecasting horizons are achieved through recursive forecasting.

The specific global methods we use are as follows.

- **Linear Autoregressive**: A linear model fit by least squares. We consider this method to be the baseline of global models. Linear models fit by least squares are important as a benchmark because they do not include any advanced machine learning technique (such as implicit regularization by SGD, bagging, or ensembling) and overlap the model class with the ARIMA model (a common local approach). Therefore, they are ideal to isolate the effect of globality.
- **Featurized Linear Autoregressive**: A linear model fit by least squares with increased complexity by adding nonlinear features in the form of polynomials of degrees 2 and 3. This is a relatively naïve way of increasing complexity, and subsumes the linear autoregressive model.
- **Deep Network Autoregressive**: Deep networks are a model class that achieves outstanding results in many applications. The architecture is a ReLu MLP with five layers, each of 32 units in width, with a linear activation in the final layer, fit by the Adam optimizer with the default learning rate. The implementation is Keras (Chollet et al., 2015). Fitting is done by early stopping on a cross-validation set at 15% of the dataset (the stacked matrix). The batch size is set to 1024 for speed, and the loss function is the mean absolute error (MAE).
- **Regression Tree Autoregressive**: Regression trees are another model class that produces great success in machine learning. The implementation is XGboost (Chen & Guestrin, 2016), with default parameters `subsampling=0.5` and `col_sampling=0.5`. Fitting is done by early stopping on a cross-validation set at 15% of the dataset. The loss function is the RMSE and the validation error is the MAE.
- **Partitioned Linear**: The dataset is initially randomly partitioned into subsets. Then, a global linear AR model is applied to each subset. This method is a simple representative of the alternative way of increasing model complexity suggested by our theory, i.e. by partitioning instead of featurization or an alternative model class.

We highlight the simplicity or naïvety of each model. We use no seasonality information, tests for stationarity, batch normalization, dropout, residual layers, advanced architectures such as LSTM or convolutional layers, attention, time dilation, ensembling, etc. No hyperparameter search is used.

Other notable families of methods, such as nearest neighbors or kernel methods, are left out for computational reasons, but we expect qualitatively equivalent results.

4.3. Errors and preprocessing

We focus on the MASE as a popular scale-free error measure which rarely divides by zero, and is therefore less prone to numerical errors than SMAPE. Furthermore, the scaling done by the MASE can be applied to data as a preprocessing normalizing step, which we apply to each series. Then, minimizing the MASE is equivalent to minimizing the mean absolute error in the preprocessed time series. Compared to SMAPE, MASE is easier to fit with off-the-shelf optimizers (it is convex), and it therefore better isolates the effect of the model class than the optimization procedure used to fit it.

4.4. Datasets

We compared the methods across a collection of datasets which have been used in many peer-reviewed publications as standard benchmarks, from local methods to recent literature for global models. We also added some examples of our own. We loosely and subjectively categorized each dataset as heterogeneous or homogeneous, based on the nature of the domains (single vs. multiple domains) and the descriptions by the original publishers.

- **M1**: Heterogeneous dataset from a forecasting competition, 1001 series, subdivided into Monthly, Yearly, and Quarterly periodicity.
- **M3**: Similar to M1, with 3003 time series and an extra "Other" category of periodicity.
- **Tourism**: Homogeneous series from a tourism forecasting competition, subdivided into Monthly, Yearly, and Quarterly data.
- **M4**: Heterogeneous dataset of 100,000 time series, subdivided into Monthly, Yearly, Quarterly, Weekly, Daily, and Hourly periodicities.
- **NN5**: From the webpage of the competition (NN5 forecasting competition for artificial neural networks and computational intelligence, 2018): The data consist of two years of daily cash demand at various automated teller machines (ATMs, or cash machines) at different locations in England. Homogeneous in data types, but heterogeneous in patterns. 111 time series.
- **Wikipedia**: Daily visits of pages within the Wikipedia domain, from a Kaggle competition (Web traffic time series forecasting, 2017). 115,084 time series of 803 observations each. Homogeneous in data type, heterogeneous patterns.
- **FRED-MD**: 107 monthly macroeconomic indicators from the Federal Reserve Bank, already differenced and log transformed according to literature consensus (McCracken & Ng, 2016). Heterogeneous.
- **Weather**: Daily weather variables: "rain", "mintemp", "maxtemp", and "solar radiation" measured at weather stations across Australia. 3010 time series from the Bureau of Meteorology and the `bomrang` R package (Sparks et al., 2020). Heterogeneous.
- **Dominick**: 115,704 time series of weekly profits of individual SKUs from a retailer. Homogeneous. From Dominicks dataset (2019), a similar dataset to Gasthaus et al. (2019).
- **Traffic**: Each series in the set is the hourly occupancy of a highway lane. Between 0 and 1. Homogeneous.
- **Electricity**: Hourly time series of the electricity consumption of 370 customers. Homogeneous.
- **Car Parts**: Monthly sales of car parts. 2674 series. January 1998 to March 2002. Intermittent demand. From the `expsmooth` R package (Hyndman, 2020).
- **Hospital**: Monthly patient count for products that are related to medical problems. There are 767 time series that had a mean count of at least 10 and no zeros. From the `expsmooth` R package (Hyndman, 2020).
- **CIF2016**: From (Štěpnička & Burda, 2017): *"72 time series with monthly frequency, from which 24 were real time series from the banking domain and 48 artificially generated. The real banking domain time series had length from 23 to 69 values and the forecasting horizon was 8 or 16 values".*. Heterogeneous.
- **Pedestrian**: Hourly pedestrian count across different streets in Melbourne. Homogeneous. 66 series ranging from 264 to 35,000+ hours.
- **Double Pendulum**: Experiment generating a chaotic time series (Asseman, Kornuta, & Ozcan, 2018). The positions of the pendulums were extracted from the video at 400 Hz. The task is to predict 200 future positions from past positions. Each time series represents a different run of the experiment or the same run sufficiently separated in time. The examples of homogeneous series all come from the same "generative process".
- **COVID19**: Per-region daily cumulative COVID19 deaths, with each of the 56 time series representing one region. The dataset was taken from the Johns Hopkins repository, featuring the first 90 days since January 22, 2020. The forecast horizon is the last 14 days.

5. Experiments

We show empirical evidence of the principles derived in Section 3 by comparing the forecast accuracy of state-of-the-art local methods to global methods. For the sake of clarity, we analyze some results from our experiments. A comprehensive summary of the experimental results is provided in Appendix.

5.1. Large memory

We first illustrate the effect on forecasting accuracy of increasing the memory of a global model, one of the principles coming from the complexity analysis in Section 3.5. We test this design principle in the most basic class of models, linear models. Given a dataset, we fit a global linear model for all possible orders of autoregression, and compare the average forecast error against the state-of-the-art local models. For each dataset, the maximum order of autoregression is the length of the shortest time series in the set, because it cannot be time-delay embedded at larger orders.

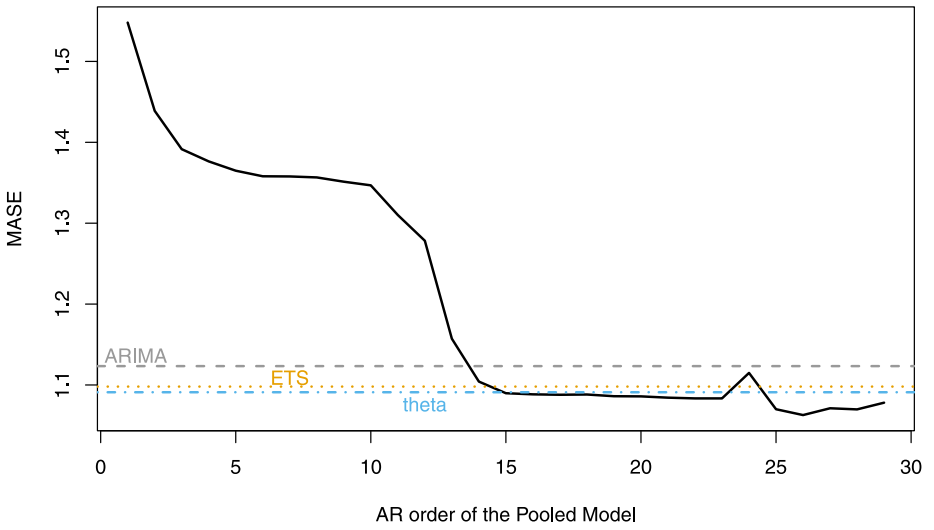


Fig. 1. The MASE of a global linear AR model as its memory increases, compared to local methods, for the M1 Monthly data. The forecast accuracy increases as the memory of the global model becomes larger, with noticeable jumps when the memory is close to seasonal periods (multiples of 12 months). A global linear model of order 25 fitted to 617 time series outperforms state-of-the-art local methods, resulting in better forecast accuracy and a simpler overall model.

We show the results for the M1 Monthly dataset in Fig. 1. We observe a clear pattern of reduction in the out-of-sample error as the order of the global AR model increases (solid black line). The global model outperforms all local benchmark methods (horizontal lines) at about AR(25). The M1 Monthly dataset has been available since the 1980s and the technology behind the global model—linear least squares—is even older. This dataset has been extensively used for testing new methods, yet a naïve global method, derived from the basic bounds, is able to outperform state-of-the-art local forecasting algorithms, which leverage sophisticated ideas from time series analysis and are based on accumulated empirical evidence. The M1 Monthly dataset has 617 time series. A single AR(25)—i.e. with 25 parameters—can "summarize" the dataset better than models that use at least one parameter per series (e.g. auto.arima needs approximately 1600 parameters to fit the whole set). This means a reduction in the number of parameters close to two orders of magnitude while achieving better accuracy. Intuitively, the M1 dataset can be considered heterogeneous; it exhibits different patterns (seasonality, long time trends, structural breaks, etc.) and represents different domains (macro, micro, demographics, etc.).

Longer memory leads global linear models to outperform local models. We extend the analysis to the rest of the datasets in the experimental setting. As mentioned above, when a dataset contains time series of different lengths, the maximum memory of a global AR model is limited to the length of the shortest time series. This artificial difficulty can be overcome with recurrent algorithms such as recurrent neural networks, but this would obfuscate the effects of the memory. To study the effects of longer memories, we modify the experiment in the following way. For a given memory level, we analyze on each dataset only the series with length above this level. This methodology is only applied to the M1 Quarterly, M3 Monthly, M4 Quarterly, and M4 Monthly datasets.

The experimental results can be seen in the Appendix. A global linear model outperforms local approaches for virtually all datasets, including all frequencies of M1, M3, Tourism, and M4 (Figure 6). It also holds for the NN5 (Figure 12), Electricity (Figure 20), Traffic (Figure 14), Hospital (Figure 19), Car Parts (Figure 18), Dominick (Figure 15), Wikipedia (Figure 21), Weather (Figure 13), Double Pendulum (Figure 22), Pedestrian (Figure 23), and COVID19 (Figure 24) datasets. The exceptions are CIF2016 (Figure 17) and FREDMD-Monthly (Figure 16); even in these datasets, the global linear model outperforms two of the three local methods.

The pattern of improving accuracy when increasing memory holds for the majority of datasets. At very long memories, we see some form of convergence in accuracy until the error explodes, because even the global model gets into an over-fitting regime. This behavior is exemplified in the Hospital dataset, Figure 19, but will happen for all datasets when the potential maximum memory is much larger than the number of series in the set. This type of over-fitting can be prevented in practice by simple cross-validation or regularization.

Notable exceptions to the pattern of "longer memory, better accuracy" are the Dominick and Car Parts datasets. We do not have a good explanation for why this happens, but both datasets comprise intermittent data. It is also interesting that very low memory orders of a global model outperform local models by a great margin.

Another peculiar effect can be seen in datasets with heavy seasonal components, such as monthly and quarterly periodicities. In these datasets, the accuracy improves with the memory, but then it stops improving or suddenly degrades at memory levels that are multiples of the seasonal frequency. After the memory level is over a seasonal frequency multiple, the accuracy substantially

improves. Fig. 1 clearly illustrates this effect. At around 10 lags the accuracy does not improve, and then it improves by almost 30% between lags 12 and 13. At lag 24 (two seasonal patterns) the accuracy experiences a "bump" and then improves again. The hourly Electricity dataset (Figure 20 showcases a similar phenomenon, the accuracy improvement stops at lag 24 (one-day cycle) and then improves again. When the memory gets close to a one-week cycle (lag 168) the accuracy degrades, and after that it gets another big improvement. We explain this effect as an over-fitting phenomenon related to the heterogeneity of seasonal strengths in the datasets. In those datasets that contain both strongly seasonal and non-seasonal series (or different seasonal periods), a global model tends to favor the seasonality at memory levels that are multiples of the frequency because it produces good in-sample results. When the memory is over the seasonal period, the global model can use its coefficients to fit both seasonal and non-seasonal patterns.

Global models fit long memory patterns. Fitting long memory patterns with local models in a completely data-driven way is difficult. For example, local models require observing several complete seasonal cycles to discover if a series has strong seasonality. Alternatively, a priori knowledge of the seasonal cycle can be added to the local model. One of the advantages of global models is the ability to pick these patterns without prior knowledge or with less data required per series.

We analyze how global models capture long memory by studying the fitted coefficients. Fig. 2 shows the coefficients of the global linear model for the M1, M3, Tourism, and M4 datasets on yearly, quarterly, and monthly frequencies. We can see that the coefficients are very similar among datasets, and the effect is consistent across datasets. The coefficients corresponding to long memory levels are large for quarterly and monthly frequencies, particularly at lags coinciding with seasonal periods, indicating that longer memories make a significant contribution to the predictions. There is a slight damping effect for longer memories. The seasonality is being automatically discovered by the global model, and is not part of the model specification. The memory orders that produce the best accuracy cannot be fitted locally without prior information. For example, to fit a basic local linear AR(40) to a series, we would need 80 observations. Realistically, we would need 10–100-times more observations than parameters in a single series to fit it by ordinary least squares. Global methods use the whole dataset to fit these coefficients.

5.2. Nonlinear models: polynomials, deep networks, and regression trees

This experiment considers increasing complexity by changing the model class of global models. We add nonlinear features such as polynomials of orders 2 and 3, and consider deep networks and regression trees instead of linear combinations of past values. Polynomials and deep networks represent an incremental or parsimonious increase in complexity with model classes that are a superset of the linear class. Regression trees represent a different model family. We test the nonlinear models for all valid orders of autoregression to isolate the effects of the model class from the effects of expanding memory.

Fig. 3 shows the effects of changing the model class on the M3 Monthly dataset. The global linear model (black solid line) improves in accuracy as its memory increases until it reaches the maximum order for this dataset (equal to the length of the shortest series). Unlike the M1 results discussed above, the global linear model applied to the M3 Monthly data does not get close to the accuracy of local benchmark methods (horizontal dashed lines). The downward trend of the error suddenly stops when we reach the maximum memory level in this dataset, suggesting that the linear model is limited in complexity; it is under-fitting. Increasing the model complexity by adding nonlinear features in the form of polynomials of degree 2 (orange line) has a small negative impact on accuracy relative to the simple linear model. A polynomial of degree 3 (pink line) has clearly better performance than the linear model.

Recursive forecasting makes polynomials numerically unstable (e.g. an 18-step-ahead forecast for this dataset is equivalent to a 54-degree polynomial) and is not recommended in the literature for automatic time series forecasting. Nevertheless, we show that even polynomials can improve results over linear models when applied globally. Regression trees (yellow line) are also more complex than linear models, but have worse performance in this dataset. They are an example of an inappropriate model class, and we can detect this in-sample through simple cross-validation. Finally, the deep network (dark blue line) also improves over the linear model, getting the best results among the global models, close to the local models in this scenario. Deep networks also suffer from the problem of instability for recursive forecasting, and the literature recommends direct forecasting (modeling each horizon separately) (Taieb, Bontempi, Atiya, & Sorjamaa, 2012), yet we show here that recursive forecasting is already competitive.

Both polynomials and deep networks demonstrate that nonlinear model classes can improve accuracy. In this example, a nonlinear model outperforms the linear models for all values of memory over 8.

We show the results of an extended experiment in the Appendix, applying the same methodology to the M1, M3, Tourism, and M4 datasets. Figure 7 shows the Yearly results, Figure 8 shows the Quarterly results, Figure 9 shows the Monthly results, and Figure 10 shows the Weekly results. Daily and Hourly results are not considered for computational reasons, but the results of the global linear can be seen in Figure 6 and Figure 11.

In agreement with the M3 Monthly results, global models such as third-degree polynomials and deep networks outperform linear models for the majority of datasets (the only exception is Tourism Yearly). Compared to local models, the nonlinear global models are able to outperform local models in the majority of datasets. The results of global models are especially strong in the large datasets of the M4 competition. In the M4 Yearly, an AR(12) polynomial of degree 2 not only outperforms the local benchmarks, but would have scored second in the

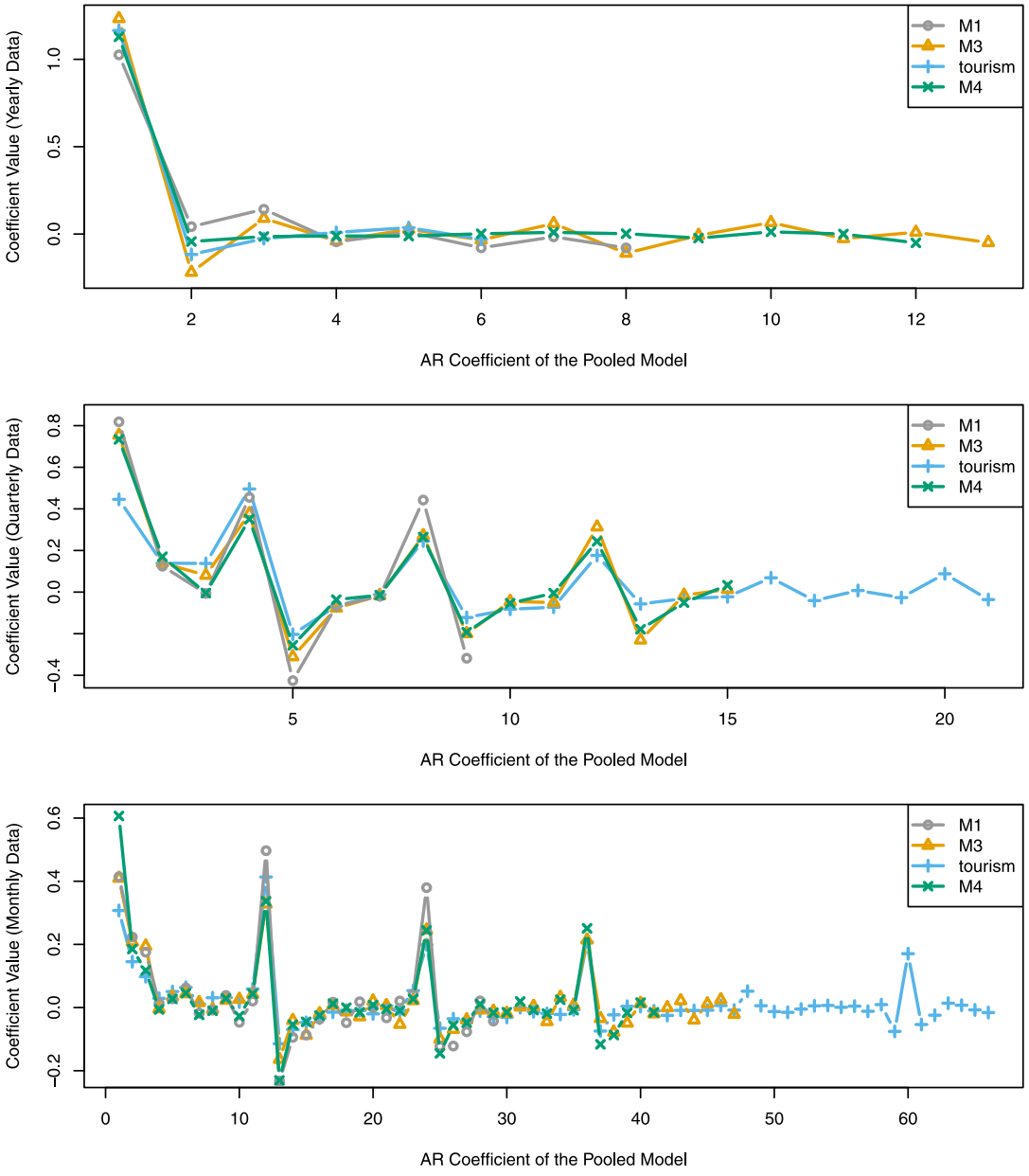


Fig. 2. Coefficients of the global linear AR model for Yearly, Quarterly, and Monthly series across the M1, M3, Tourism, and M4 datasets. The fitted coefficients are similar for all datasets. The global linear AR model has large coefficients for long memory levels for both Quarterly and Monthly data.

M4 competition according to its MASE of 3.01. This means that 24 numbers that specify the model can forecast the 23,000 time series of the M4 Yearly better than most of the methods that participated in the competition. For the M4 Quarterly, the deep neural network would have achieved the third-best ranking.

We interpret these results as strong empirical evidence towards globality for the design of new learning algorithms. We can increase the complexity of a baseline global model (linear) in naïve ways (polynomials), and a learning algorithm such as least squares can succeed in finding better forecasting functions. Modern machine learning algorithms such as deep networks produce outstanding results consistently across multiple datasets.

These results should not be interpreted as a recommendation or preference for a specific model class over the others. For example, regression trees could perform better with some hyperparameter tuning.

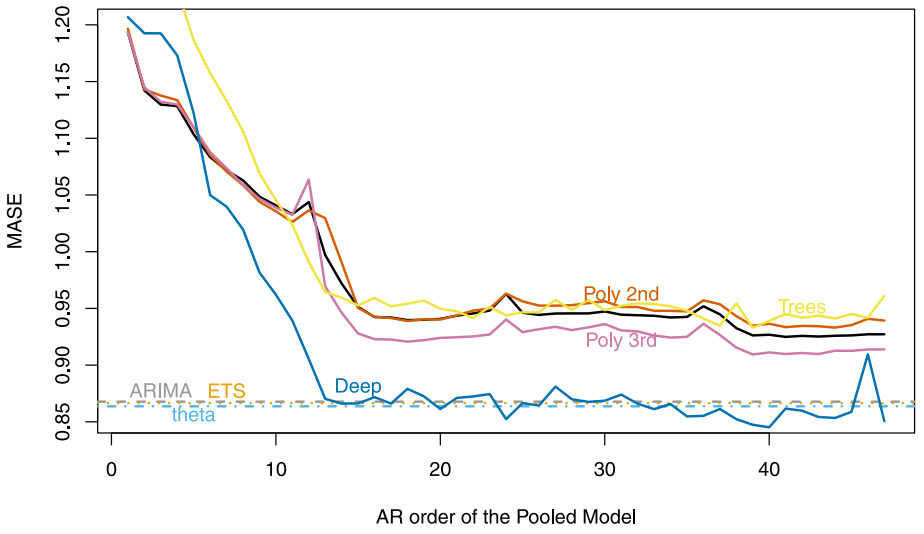


Fig. 3. The performance of nonlinear autoregressive models on the M3 Monthly data. Solid lines are the global methods, horizontal dotted lines are the local methods. Black: baseline global linear model; orange: polynomial of degree 2; pink: polynomial of degree 3; yellow: regression trees; dark blue: deep network. The M3 Monthly dataset contains 1428 time series. The global linear model increases in accuracy as the AR order increases, but it is limited by the maximum order possible in this dataset. Increasing complexity through polynomials is effective for polynomials of degree 3, which improve accuracy over the baseline global linear model for almost all AR orders. Regression trees do not improve over the linear model, although the performance also increases with memory. The deep network improves the most over the linear model, reaching a level of accuracy comparable to that of the local model. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Model complexity vs. memory In terms of the effect of memory, we highlight two phenomena. The main observation is that both sources of complexity, memory and model class, are effective at improving forecast accuracy. When both are included, the model class interacts with memory in different ways depending on the dataset, but in general a more complex model class requires less memory to achieve the same level of accuracy as the more simple class.

5.3. Partitioning

We experimented with another method of increasing complexity: partitioning the set of series. We split the dataset *randomly* into 10 groups of equal size, and each group was fitted and forecasted by a global model.

The implications of the experiment shown in Fig. 4 are quite interesting, since partitioning is equivalent to *reducing* the sample size, something not usually recommended in statistics and machine learning. Since partitioning is done at random (not clustering), it is strong evidence towards the benefit of raw complexity in a model rather than finding the "right" model, and it shows that having similar series is not the cause of the good performance of global methods. Beyond the possibility of more accurate results, partitioning might also result in faster training times through "embarrassing" parallelism.

More examples of this effect for the M1, M3, M4, and Tourism data can be found in the Appendix (Figure 11), but the effect holds (with more or less beneficial impact) in the majority of datasets where the global model is complexity-constrained.

The experiments in this section are meant to showcase the strength of theoretical results because they predict a counter-intuitive effect. They are not a specific recommendation.

5.4. In-sample vs. out-of-sample loss

This experiment shows that the ability of simple local methods (e.g. ets) to approximate the data generating processes is not the main limiting factor of their performance in many practical situations. The superior accuracy of global methods is more likely attributable to their better generalization. This can be considered one of the main predictions from the theory: when we introduce some complexities for specific local and global algorithms in the bounds of Proposition 2, we find that the aggregate complexity of local models is higher than that of global models.

We can expect the following.

- Local models should have a smaller in-sample error than global models for comparable model classes (e.g. in the linear case, auto.arima, and the global linear model).
- Higher complexity implies higher risk of over-fitting. We should see more over-fitting from local models (difference between in-sample and out-of-sample errors).

We compared the average in-sample loss that local and global methods produce against their respective one-step-ahead out-of-sample loss for the M1, M3, Tourism, and M4 datasets. We used in-sample loss as a measure of the capacity of a given model class to approximate the data.

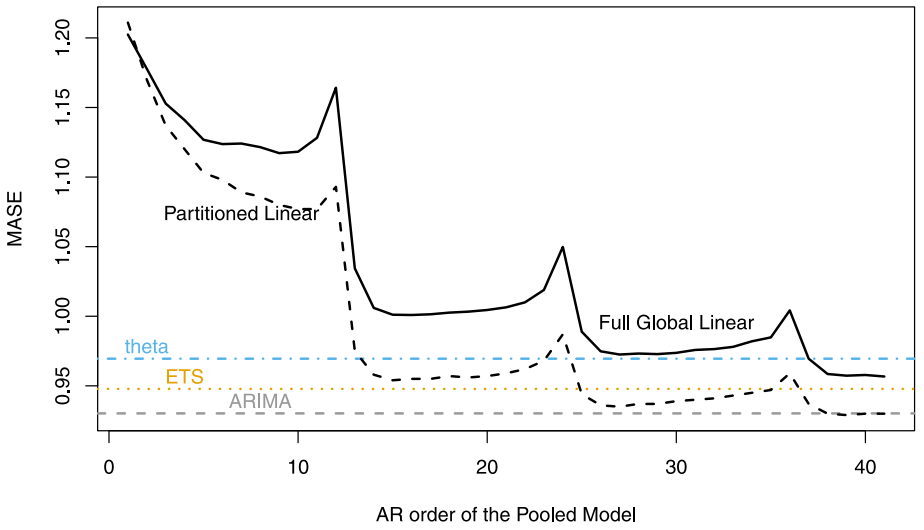


Fig. 4. The MASE of global linear models when complexity is increased by partitioning the set. The M4 Monthly is randomly partitioned into 10 parts of equal size, and a separate global model is applied to each subset of the partition. Solid line: MASE of a global linear AR; black dashed line: a global linear AR after partitioning. The partitioned linear models outperform local methods and the non-partitioned global model. An AR of order 20 applied on each partition is already good.

It measures how well an ideal function within the model class can fit the data, even though in practice we might not be able to find this function. We focused on one-step-ahead forecast loss, instead of longer horizons, because it is the measure that both local and global methods minimize when training. The theoretical results relate training to testing losses when they measured the same quantity, and at the same time we removed the confounding factors of longer horizons.

When a model is limited in capacity, it is not able to fit the data well, so its in-sample error is large. We see in Fig. 5 the average in-sample error (light bar) superimposed over the out-of-sample error (darker bar) for each method and each dataset. The difference between these two bars is the generalization error. Local models have better average in-sample loss than the global alternatives. Local models suffer from over-fitting, rather than from being unable to fit complex series. The global linear model exhibits the smallest generalization error, a result that can be predicted by the theory, since it is by far the simplest of the model classes. Even a global deep network has less difference between in-sample and out-of-sample errors than local models, although it is a quite complex model class.

The more common explanation for global models (see Section 6.1) is that classical local models cannot pick "complex patterns" in the data. However, this experiment suggests a different explanation for the empirical performance of global and local methods. The more likely reason for the performance is the over-fitting of local models; we can fit the series well with simple local models but do not know how to control their complexity. For global methods, standard complexity control techniques are more successful or not even needed.

5.5. Heterogeneous datasets

We show empirical evidence of the capacity of global methods to adapt to heterogeneous datasets by increasing the complexity of the model class. We consider the *full* M4 dataset as the heterogeneous set for our tests. When we take the M4 as a single set of time series, we expect is to be highly heterogeneous, because it contains 100,000 time series from different domains (measuring different units), and at different frequencies (from hourly to yearly measurements).

We compare the average out-of-sample error of the one-step-ahead forecasts for the local benchmark methods and the global linear and deep network, fixed at a memory level of AR(12). We are limited by the length of the shortest series in the set. We add two additional global models that introduce information about the heterogeneity.

- The first model clusters the data by its frequency before fitting a global model to each cluster.
- The second model adds information about the frequency and other features (Hyndman, Kang et al., 2020) as extra features of the model in addition to the last 12 observations, but it does not partition the dataset.

The purpose is to compare "homogeneous manual clusters" with "heterogeneous with some manual information". The dataset with the added features is still heterogeneous, and all extra features are functions of the data (except the frequency).

We can see in Table 1 that the "plain" global models are outperformed by the local models. Even though they are overshadowed by local models, this result is already

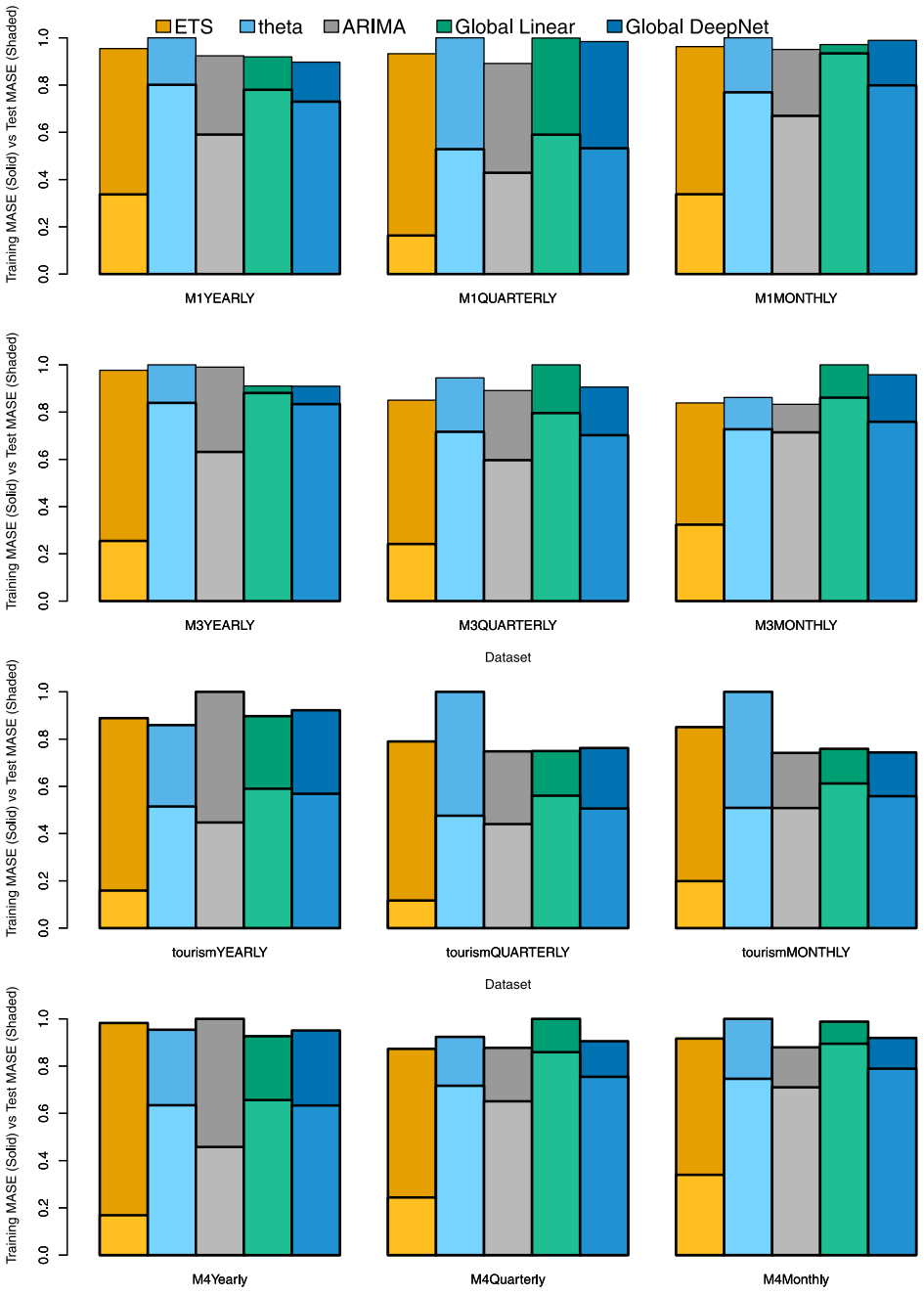


Fig. 5. Generalization error of local and global methods in the M1, M3, Tourism, and M4 datasets, measured as the in-sample MASE in lighter shaded bars, and superimposed over their respective out-of-sample MASEs in darker bars, on one-step-ahead forecasts. State-of-the-art local models (ets, ARIMA, theta) tend to have better in-sample loss than global models, suggesting they do not suffer from under-fitting. However, the generalization error of global models is superior to the local alternatives (in-sample and out-of-sample errors are closer). Even a deep network with thousands of parameters has a worse in-sample error but is competitive out-of-sample. Global methods are in fact simpler than local at per-series complexity.

interesting because global models outperform naïve forecasting by a large margin (a naïve seasonal method would have an MASE of roughly 1), something that is not trivial. In particular the global linear is fitting 12 numbers to 100,000 time series, more than four orders of magnitude fewer parameters that an ARIMA.

When we cluster the datasets, the global models substantially improve performance. In particular the deep network already outperforms two of the three local models. These results are to be expected because we know from other experiments that global models outperform local models in the M4 for each individual frequency, but here we are severely limiting them in terms of memory. When we add the extra features, we see that the deep network improves accuracy, outperforming all local and the clustered global. We see this result as strong

Table 1
MASE of one-step-ahead forecasts in the M4 dataset for local, clustered-by-period, and fully global methods. Clustering by periodicity reduces the heterogeneity of the dataset and improves forecast accuracy over naïve global methods. Augmenting a fully global neural network with features produces equivalent results to clustering.

Method	MASE
auto.arima	0.7618
ets	0.7644
theta	0.7745
Global linear	0.9279
Cluster linear	0.8559
Global DeepNet	0.8149
Cluster DeepNet	0.7632
Glob. Deep+Feat.	**0.7596**

evidence towards the power of complex model classes for global models; they can automatically find good forecasting functions even in very heterogeneous datasets.

5.6. Scale normalization as preprocessing

We compare the effect of scaling as a means of preprocessing the dataset before applying global models, to show that scale information can be relevant. Scale normalization is one of the main forms of preprocessing applied in global models. However, one of the benefits of global models is precisely that they offer a way of modeling information coming from scale in a data-driven way, when local models only have information within each series. For example, a global model can learn that time series in the Traffic dataset only take values between 0 and 1. But this information might not exist within an individual Traffic time series, so it should be manually added to local models.

Scale invariance is appropriate in most benchmark datasets because they have been arbitrarily re-scaled or mix series with different units (e.g. tourist arrivals and unemployment rates). Moreover, popular error measures (MASE or SMAPE) are scale invariant, artificially emphasizing the impact of scale normalization. We expect real applications to exhibit none of these issues. Since there seem to be no real reason for scale invariance, and some good reasons for not normalizing (or removing scale information), it is interesting to experiment with scale invariance.

In order to keep the "best of both worlds" we compare a method that does scale normalization against a method that scale-normalizes but adds the scale of each series back as an additional variable to each series. These two methods are compared to a third that does not normalize the data at all. We use the global deep network of AR(12) (we cannot use linear methods because they are scale invariant) as the model for all three methods. The only aspect that changes is whether scale-normalization preprocessing is added. For each of the datasets we measure the error in a one-step-ahead forecast horizon. Table 2 shows average errors, both in MASE and plain mean average error (MAE). The errors of the scale-normalized and the "scale-normalized with scale features" datasets are shown relative to the non-scaled version: the error for each dataset is divided by the respective error of the non-normalized method and then averaged across all datasets.

Table 2
Forecast errors of two types of normalization preprocessing relative to the non-normalized version and averaged across datasets. The model is a deep network of AR(12). Adding scale information has a small positive impact in accuracy in both MASE and MAE errors, measured relative to the error produced by a raw non-normalized deep network.

	Normalization	Normalization + scale as feature
Relative MASE	0.9497	0.9479
Relative MAE	1.0213	0.9959

We can see that scale-normalization has a positive effect relative to the non-normalized version (errors smaller than 1 indicate better performance than the non-normalized version). Furthermore, adding scale information as an additional feature has a further positive effect in reducing the error. The non-normalized version also captures scale information but is very sensitive to the influence of series in the set with higher scales, because of their weight in the loss function. This experiment shows that scale information can be meaningful, but is not a definitive prescription on how to model it. Specific ways of capturing it will require further work or fine-tuning, for example in the network architecture or loss function, which might allow for better performance on the non-scaled version.

6. Discussion and related work

6.1. Global and local

There is a rich literature on forecasting groups of time series dating back to at least 1962 (Zellner, 1962), where a form of data pooling is studied. We refer the reader to the excellent paper by Duncan et al. (Duncan, Gorr, & Szczypula, 2001) for an introduction to the key elements in the literature.

Global methods for forecasting sets of time series can be classified into "classic" and "modern" categories. The classic methods include those based on the use of state-of-the-art, well-tested models in combination with data pooling, exemplified by Duncan et al. (2001) and Trapero, Kourentzes, and Fildes (2015). Classic methods argue for the use of global models but without increasing the complexity with respect to local models (resulting in simple global models), and support this claim with empirical evidence. While this prescription is true in some scenarios (e.g. Trapero et al. (2015) use a global model only for short time series where a basic model is already complex), our work gives a theoretical motivation in the *opposite* direction. Global models can be more complex than individual models in a local algorithm and still benefit from better generalization. Empirical evidence for this claim appears in Section 5.

Modern methods include those based on the use of recent machine learning models (e.g. deep nets) as a global model. Exemplified by Salinas et al. (2020), Laptev et al. (2017), and Bandara, Bergmeir, and Smyl (2020), global models are prescribed with higher complexity than the

state-of-the-art local individual models, achieving outstanding empirical performance, though lacking theoretical motivation. Our work serves as a theoretical justification for this prescription: a local algorithm applied to a dataset increases in complexity with the number of series, so a global model should increase its complexity to keep up.

Our results also extend the prescriptions in modern approaches. The main motivation for the use of complex models given in Salinas et al. (2020) and Laptev et al. (2017) is the ability of complex models to pick "complex patterns" (e.g. "nonlinear feature interactions") shared across series in the set. This again might be true in some scenarios, but our work suggests a more comprehensive rationale. In practice, local-and-simple models are able to fit the data better (i.e. with lower in-sample loss) than global-and-complex models. In other words, pattern complexity is not a problem for time series forecasting (unlike other learning problems): a local linear model with a few parameters can produce results so accurate as to be considered unbelievable. If we want to estimate a complex pattern in a series in ideal conditions of sample size, then we are better off with local models. The (often better) performance of global-and-complex models is more likely due to a better generalization; patterns picked by a global-and-complex method are in fact simpler than the local-and-simple method when taken individually. Evidence in Section 5.4 shows that the in-sample error of local models is lower than in complex global models, implying that they can fit the data better. Therefore, the prescription in literature on modern approaches is limiting in the sense that global-and-complex models can be used not only when the set of series involves similar complex patterns. Our theoretical justification shows that it can also be used when there are many unrelated simple patterns (which will happen more often).

Another highlight of our analysis is the emphasis on memory. We showed that global models require more memory than local models to guarantee the same approximation capacity. This contrasts with the classic global models that assume that all time series follow the same model as the local alternative. Longer memories are required for global models even when they have other means of increasing complexity over local alternatives (e.g. by adding nonlinearities).

6.2. General applicability of global models

Under the traditional statistical learning framework of consistent estimators and true data generating processes, global methods should not be used because they cannot approximate as well as local models. A synthetic scenario can be carefully crafted in which *a* global method performs much worse than *a* local method. Proposition 1, by changing the setting to a more realistic one (finite data and unknown ground truth), challenges the applicability of the traditional framework. Proposition 1 shows that, ideally, global and local methods are able to produce the same forecasts. It is also specific to time series forecasting, since this does not happen for regression tasks in general. The opposite is also true: the property of the consistency of an estimator (or "universal approximator" of neural networks) cannot be invoked as a rationale for the use of global models in the traditional setting (no single function exists which can approximate a set of time series in general).

Even though global models were being used years before the motivation from Proposition 1, the value of this theoretical result goes beyond being an interesting curiosity, because the restrictive traditional framework is still implicitly carried into practice. For example, the literature on global models encourages their use for groups of time series which are similar or analogous, but not for general sets. From Proposition 1 we can derive that global models can approximate general sets as well as local models, without requiring similarity or other "tricks" such as adding dummy variables to identify each series in the set to be able to fully pick the patterns, as in Salinas et al. (2020) (Section 3.4) or (Oord et al., 2016). While these techniques can and will lead to better performance in some scenarios, they also limit generalization and interpretability. So it is interesting to know that they are not a strict requirement for approximation. For example, adding dummy variables identifying each series in the set to a global linear model with interactions is equivalent to a local linear model.

Proposition 1 can be exploited further because it is valid for any (practical) set of time series, which in fact means that successful global models can be applied even to single series by artificially adding *any* series to form a set. A clear consequence is data augmentation: series can be added to a set while guaranteeing that a global algorithm exists which fits the augmented set as well as a local model fits the original set.

We hope that Proposition 1 will enable the exploration of ideas that may have been subconsciously inhibited by the restrictive traditional framework, encouraging a deeper study of global models and the proposal of new global learning algorithms.

6.3. Partitioning, relatedness, and clustering

Global models have been introduced to address the problem of forecasting sets of series which are "related", "analogous", or have "similar patterns", but not for general sets (Bandara et al., 2020; Duncan et al., 2001; Salinas et al., 2020). This notion of relatedness has been introduced mostly through examples, but never formally defined. In order to achieve or increase the relatedness of a set, several notions of "clustering by relatedness" have been introduced. The most prevalent clustering type is judgmental clustering (Duncan et al., 2001). Judgmental clustering is applied manually and is based on the natural characteristics of the series, such as what units of measure are used (sales of products and precipitation series belong to different groups), or the frequency of the series (monthly, daily, or quarterly series in separated groups). Algorithmic clustering (as in as *k*-means or partitioning against medoids) based on features of the series (e.g. autocorrelation coefficients) have also been applied (Bandara et al., 2020). Both forms of clustering often result in improved forecasting accuracy, but their

nature is mostly intuitive or heuristic (there is no general guarantee that they affect forecasting accuracy). And insofar as they impose a reduction in sample size, they can also be detrimental.

Our work can serve to analyze relatedness and clustering. First, from Proposition 1 we show that relatedness, and therefore clustering, is not essential to forecasting sets. We can think of clustering+modeling as another type of model. The bounds of Proposition 2 and Section 3.6 show clustering as a form of set partitioning and inform us about the trade-offs compared to other methods (such as purely local or purely global methods).

Second, the formalism of Proposition 2 and Section 3.6 allows us to define a concrete notion of relatedness. The elements involved in relatedness are clearly identified: a partition of a set, a learning algorithm (including the model class), and a loss function. We cannot measure the relatedness of a partition without specifying the forecasting model. Given a set of time series and two ways of partitioning it, say P_1 and P_2, partition P_1 is more related than P_2 if its bound is better. When P_1 and P_2 have all sub-elements of equal size, it reduces to which has better in-sample error.

This view of partitioning links to the main formulation on multi-task learning (Thung & Wee, 2018), with each of the time series representing a task. It is an extension of partitioning or clustering: prior information on task similarity is expressed through a weight matrix, a weighted graph which can represent every partition from global to local, clusters of the set, and even soft clustering. From this point of view, we can see clustering as a "layer" or preprocessing step in our architecture that infers this weight matrix either manually or from features. Compared to increasing the complexity of the model, clustering has less flexibility and does not directly minimize the loss function. In a way, any extra information (either judgmental or features) that is considered by the clustering step can be added via data augmenting to a fully data-driven, end-to-end algorithm, and we can let the data speak for itself. In Section 5.5, we showed that a global deep network model is able to produce good forecasts for a highly heterogeneous dataset without requiring judgmental or algorithmic clustering; it is in fact able to fit the data surprisingly well. With the methodological advances in machine learning, computing power, and the increase in the availability of data, we expect that general-purpose machine learning models will get diminishing benefits from adding ad hoc clustering methods on top of them.

6.4. Modern practices and model classes for global methods

We now provide some critical analysis of specific recommendations in "modern" approaches based on our results.

- **Model classes**: Most "modern" methods prescribe the use of a specific deep network architecture. Our work decouples the complexity from a particular model, and therefore motivates and justifies the use of alternative model classes (e.g. trees in Ma and Fildes (2020), featurization, kernel methods, or nearest neighbors). The justification increases the "toolbox" available to practitioners. For example, gradient boosted decision trees also benefit from the theoretical properties presented here, and have been known to produce very good results in forecasting competitions, such as the GEFCom2014 (Landry, Erlinger, Patschke, & Varrichio, 2016) and the recent M5 (M5 forecasting competition, 2017), in particular when adding external information.
- **Memory**: Many global models are based on long short-term memory (LSTM) or other recurrent deep network architectures. Despite their name, these methods have difficulties learning long memory patterns without manual tuning (Trinh, Dai, Luong, & Le, 2018). One common way of tackling this limitation is to use a larger input window, making the methods more similar to a purely autoregressive network. We showed that even non-recurrent, global AR models can successfully fit long explicitly specified memory orders, and we expect that the concept of really long input windows within a global method combined with a recurrent architecture can produce benefits while retaining the good properties of recurrence. In Section 5.1 and the Appendix we show the effect of long memory on forecasting accuracy. Other recent architectures such as temporal convolutions with time dilation (Borovykh, Bohte, & Oosterlee, 2018; Oord et al., 2016) also go in the direction of fitting really large memory levels. Convolutions, on the other hand, are successful because they substantially reduce the capacity of the model class, so there is the danger of under-fitting when we move towards larger, more heterogeneous datasets.
- **Scale normalization as preprocessing**: Several forms of preprocessing have been recommended for global methods (Duncan et al., 2001; Hewamalage, Bergmeir, & Bandara, 2021; Rabanser et al., 2020; Salinas et al., 2020; Smyl, 2020). They are considered an important component of the performance of the methods. The main such preprocessing step is normalizing the scale of each series: each series is transformed by removing its scale before model fitting and forecasting, and then the forecasts are back-transformed. Scaling can help deal with outlier series (e.g. one series has a much larger scale than the rest, dominating the fitting) and in the case of deep networks, scaling makes them faster to train.
  This form of scaling the series locally makes *any* model scale invariant, a strong assumption which reduces the complexity of the model. Interestingly, it is one of the two properties that makes linear maps. The trade-off therefore is between how relevant the information about scale is compared to how it reduces over-fitting or deals with outliers, but in general we know that time series forecasting is not scale invariant. For example, the series in the Traffic dataset have a clear maximum value of 1 implying maximum occupancy in the lane. When series in this dataset take values close to this maximum we know that they cannot go beyond that value. This

is valuable information for forecasting. After scaling, series close to the maximum value may look exactly the same as those in the middle range, so this relevant information is lost. On the other hand, there are examples of processes which would clearly benefit from scale normalization. For example, if our set of time series comprises exponential growth patterns, each time series can be scale-normalized without loss of information, and the pattern will be much easier to pick for a global model.

Coincidentally, the errors commonly used for benchmarking (such as SMAPE or MASE) are scale invariant, so this form of preprocessing also helps optimize the correct loss function. However, many real situations will not need these scale invariant error measures, so scale invariance becomes a way of reducing complexity. As we showed, global models can afford to be quite complex, so it is worth exploring alternative ways of dealing with specific problems that are solved by scale normalization (such as the weight of outliers, training speed, etc.) rather than making everything scale invariant. An experiment on the effect of scaling can be found in Section 5.6, showing that although scaling can convey important information in some cases, it is not a definite argument on the use of normalization.

- **Seasonality**: Another form of preprocessing is modeling seasonal patterns individually, subtracting them from each series and then modeling the residuals with a global method. A rationale for this form of preprocessing can be found in Smyl (2020): "Neural networks do not learn different seasonality" (page 4) and in Bandara, Bergmeir, and Hewamalage (2019). Our empirical results suggest that global neural networks do not suffer from an inability to pick seasonal patterns, and the theoretical justification of previous empirical evidence suggests that the models used there are not sufficiently complex or have limited memory. Removing seasonality from the data can erase some relevant information which might be exploited by global models, as with scale normalization.

6.5. Multivariate, covariate, and sequence models

There is a large overlap between global models and multivariate forecasting models, because both work on groups of time series. In practical terms, global models are univariate time series models. They only require observations of a single time series for forecasting, and they use the group to find good predictive functions. Consequently, global models are more generally applicable because they do not require observations from multiple time series *at the time of forecasting*. Multivariate models work on groups that are supposed to have some form of interaction or joint evolution. Global models work on any group. For example, forecasting a pandemic with a global model can benefit from time series measured at different historical periods (separated by decades), controlled experiments, or even different pathogens, while a multivariate model is limited to the evolution of the current one and the factors that influence it. Another example could be electricity consumption patterns. A global model could pool data from different places around the world to find a shared pattern, while it is clear that completely unrelated households at different corners of the world will not benefit from a multivariate approach that uses their joint evolution.

When there is inter-dependence (multivariate) or dependence (covariate and a main time series) in the dataset, global methods do not capture it directly, though they could help capture it in an implicit manner, by the same reasoning that allows time series to be predicted from their own past observations (time-delay embedding) even when they depend on other unobserved processes. This effect is of course not unique to global models, but they can contribute in that they can help pick more complex patterns than local alternatives.

From a more theoretical point of view, the generalization of global models that we show here assumes groups of independent time series. Under heavy dependence, global models lose their beneficial performance guarantees (though practitioners could still try them). Multivariate methods specifically deal with this strong dependence case by heavily reducing the complexity, either by considering that the joint group of time series can be represented at a much lower dimensionality or through other forms of regularization (Sen, Yu, & Dhillon, 2019; Wang et al., 2019).

Ultimately, globality and multivariateness are orthogonal concepts, globality representing the "same pattern of evolution" and multivariateness representing "joint or dependent evolution". For example, one can have a multivariate global model, i.e. a group of different multivariable processes that are assumed to evolve in the same way. For example, pairs of energy consumption per household and income per household time series represent a multivariate process. We can have a dataset with many of these multivariate processes, for example by sampling households all around the planet. Each multivariate process does not necessary affect the other, but we could assume that they follow the same pattern. Our results can even be adapted to this more general case by assuming that all time series in a multivariate joint has a finite support, so the multivariate can be encoded in a single number producing a univariate time series.

The main property of the equal approximation of global and local methods that we show for the univariate case will be inherited by the multivariate global case, and more interestingly by the case of univariate time series with covariates. The latter case highlights the difference between time series and other regression scenarios, even sequence-to-sequence. If we want to forecast a main sequence based only on other sequences, without using the main sequence series as input to the prediction function, we are in a standard regression and there is no guarantee that global models will be able approximate as well as local models. For example, we might want to predict sales of ice cream based on temperature data and sales of umbrellas based on temperature data by pooling both time series in a global model. It is clear that these two time series are different functions of the temperature, so

a global model will not be able to approximate as well as a local one. When we add the main time series as the extra input, we get the equal approximation capacity, but the benefit comes from the principle that we show in Proposition 1 for the univariate case.

7. Conclusions

We showed that global and local methods for forecasting sets of time series are equally general. Globality is neither restrictive nor requires "similarity" or "relatedness" in the set. We then compared the relative complexities of local and global methods, showing that global methods can afford to be more complex than local methods because they generalize better. The combination of these two findings strongly supports recent empirical results and motivates further exploration of modern machine learning methods for time series forecasting.

The theoretical symmetry between global and local models can be broken, however, by considering some practical issues.

- While the better generalization of global models does not imply better performance, it is already useful because it makes it easier to detect when an algorithm fails (e.g. by cross-validation or simply because in-sample loss does not reach satisfactory levels). Arguably, this benefit is even more important in time series forecasting than in other more stable machine learning problems (see Makridakis, Spiliotis, and Assimakopoulos (2018), Table 10).
- Global models can increase model complexity compared to individual local models. Modern machine learning has developed an impressive array of techniques for increasing and controlling model complexity (e.g. deep networks, regularization) that can be applied to global models. Meanwhile, finding the right simple models to fit locally requires domain knowledge or prior information, which can be difficult to obtain. On the other hand, when prior information is already available, we can usually express it only through local models.

The study of global models for time series forecasting is a relatively new field, and there is an opportunity for researching ways of expressing prior information for global models. Ultimately, the seemingly superior performance of global models is a question of context: most problems for which we have simple, domain-based local models are not usually considered for automatic forecasting because they are already solved. For example, we use automatic forecasting for sales because of the lack of domain understanding, but for weather forecasts we have domain-based models, so we do not consider this dataset for benchmarking data-driven univariate forecasting.

Our complexity analysis comes from the derivation of basic generalization bounds. The bounds can be used to identify the main elements involved in the generalization of globality and locality at a general level. From the bounds, we derived three parsimonious principles for designing new algorithms. The first principle points towards the increasing complexity of model classes for global methods (nonlinearities, deep networks, etc.) and serves as a strong motivation and support for recent results. The second principle of increasing the memory for global methods has strong implications for forecasting by showing that global models might be able to incorporate larger memory much more effectively. The third principle of partitioning a group of series bridges the gap between local and global approaches, and serves to link the related literature, highlighting the hidden costs of methods such as clustering. Besides the basic principles, the proposed framework can be used to analyze concepts such as relatedness, for specific architectural decisions such as scale-normalization, recurrence, or manually added seasonality, and to extend the interpretation of the success of global methods to more heterogeneous settings.

Because of their generality, the bounds are necessarily loose; their benefit comes from the insights derived from their analysis. Tools are needed to navigate the space of all possible models or architectures *before we see the data*. Otherwise, it becomes impossible to do it efficiently through trial and error, leaving us at the mercy of spurious results. For tighter bounds, particularly in time series where the effective sample size is difficult to estimate, we recommend cross-validation (Bergmeir, Hyndman, & Koo, 2018). Cross-validation is powerful at the global level, and the current state of technology relies on testing many different models and hyperparameters, which is safe to do globally (unlike locally).

We presented ample empirical evidence comparing global and local models and the algorithms we suggested. We showed that a parsimonious increase in the complexity of global models makes them highly competitive, outperforming local state-of-the-art models on most datasets. In our estimation, the strongest empirical claim comes from the linear case. While outperformed by advanced methods such as deep networks, global linear models achieve comparable or superior accuracy to state-of-the-art local models, yet require *orders of magnitude fewer parameters than the competing local alternatives*. Such models are simpler and more accurate. There is no danger of data snooping or cherry picking, as we are showing every possible linear model that can be fitted to each dataset, and the results present a consistent picture. Linear models are well understood. It is trivial to create situations where global linear models fail, but it seems that these situations are difficult to find in practice. From this fact, it is easy to believe that a global model that is slightly more flexible than linear will produce even better results. The strength of the empirical results challenges the notion of the "relatedness" or heterogeneity of datasets with thousands of time series such as the M4.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the Australian Centre of Excellence for Mathematical and Statistical Frontiers. We thank the anonymous reviewers for their suggestions, particularly for expanding the scope of Section 2.

Appendix A. Supplementary data

Supplementary material in the form of figures referenced in the paper can be found online at https://doi.org/10.1016/j.ijforecast.2021.03.004. Numerical results can be found following the instructions at https://github.com/pmontman/paper-global-forec-princip.

References

M5 forecasting competition. (2017). https://www.kaggle.com/c/m5-forecasting-accuracy.

Abu-Mostafa, Yaser S., Magdon-Ismail, Malik, & Lin, Hsuan-Tien (2012). *Learning from data, Vol. 4*. AMLBook New York, NY, USA.

Asseman, Alexis, Kornuta, Tomasz, & Ozcan, Ahmet (2018). Learning beyond simulated physics. In *Modeling and decision-making in the spatiotemporal domain workshop*.

Bandara, Kasun, Bergmeir, Christoph, & Hewamalage, Hansika (2019). LSTM-MSNet: Leveraging forecasts on sets of related time series with multiple seasonal patterns. arXiv preprint arXiv:1909.04293.

Bandara, Kasun, Bergmeir, Christoph, & Smyl, Slawek (2020). Forecasting across time series databases using recurrent neural networks on groups of similar series: A clustering approach. *Expert Systems with Applications*, *140*, Article 112896.

Bergmeir, Christoph, Hyndman, Rob J., & Koo, Bonsoo (2018). A note on the validity of cross-validation for evaluating autoregressive time series prediction. *Computational Statistics & Data Analysis*, *120*, 70–83.

Borovykh, Anastasia, Bohte, Sander, & Oosterlee, Cornelis W. (2018). Dilated convolutional neural networks for time series forecasting. *Journal of Computational Finance*, *22*(4), 73–101.

Chen, Tianqi, & Guestrin, Carlos (2016). Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 785–794).

Chollet, François, et al. (2015). Keras: Deep learning library for theano and tensorflow. https://keras.io.

Dominicks dataset. (2019). https://www.chicagobooth.edu/research/kilts/datasets/dominicks.

Duncan, George T., Gorr, Wilpen L., & Szczypula, Janusz (2001). Forecasting analogous time series. In J. Scott Armstrong (Ed.), *Principles of forecasting* (pp. 195–213). Springer.

Gasthaus, Jan, Benidis, Konstantinos, Wang, Yuyang, Rangapuram, Syama Sundar, Salinas, David, Flunkert, Valentin, et al. (2019). Probabilistic forecasting with spline quantile function RNNs. In *The 22nd international conference on artificial intelligence and statistics* (pp. 1901–1910).

Hardt, Moritz, Recht, Ben, & Singer, Yoram (2016). Train faster, generalize better: Stability of stochastic gradient descent. In *International conference on machine learning* (pp. 1225–1234).

Hewamalage, Hansika, Bergmeir, Christoph, & Bandara, Kasun (2021). Recurrent neural networks for time series forecasting: Current status and future directions. *International Journal of Forecasting*, *37*(1), 388–427.

Hoeffding, Wassily (1963). Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, *58*(301), 13–30.

Hyndman, Rob J. (2020). expsmooth: Data sets from "Exponential smoothing: a state space approach" by Hyndman, Koehler, Ord and Snyder (Springer, 2008). R package version 2.4. http://pkg.robjhyndman.com/expsmooth.

Hyndman, Rob J., Athanasopoulos, George, Bergmeir, Christoph, Caceres, Gabriel, Chhay, Leanne, O'Hara-Wild, Mitchell, et al. (2020). forecast: Forecasting functions for time series and linear models. R package version 8.12. http://pkg.robjhyndman.com/forecast.

Hyndman, Rob J., Kang, Yanfei, Montero-Manso, Pablo, Talagala, Thiyanga, Wang, Earo, Yang, Yangzhuoran, et al. (2020). tsfeatures: Time Series Feature Extraction. R package version 1.0.2. https://pkg.robjhyndman.com/tsfeatures/.

Hyndman, Rob J., & Khandakar, Yeasmin (2008). Automatic time series forecasting: The forecast package for R. *Journal of Statistical Software, Articles*, *27*(3), 1–22.

Jaganathan, Srihari, & Prakash, P. K. S. (2020). A combination-based forecasting method for the M4-competition. *International Journal of Forecasting*, *36*(1), 98–104, M4 Competition.

Kuznetsov, Vitaly, & Mohri, Mehryar (2016). *Forecasting non-stationary time series: From theory to algorithms*: *Technical report*, https://cims.nyu.edu/~vitaly/pub/fts.pdf.

Kuznetsov, Vitaly, & Mohri, Mehryar (2017). Generalization bounds for non-stationary mixing processes. *Machine Learning*, *106*(1), 93–117.

Landry, Mark, Erlinger, Thomas P., Patschke, David, & Varrichio, Craig (2016). Probabilistic gradient boosting machines for GEFCom2014 wind forecasting. *International Journal of Forecasting*, *32*(3), 1061–1066.

Laptev, Nikolay, Yosinski, Jason, Li, Li Erran, & Smyl, Slawek (2017). Time-series extreme event forecasting with neural networks at Uber. In *International conference on machine learning, Vol. 34* pp. 1–5.

Ma, Shaohui, & Fildes, Robert (2020). Forecasting third-party mobile payments with implications for customer flow prediction. *International Journal of Forecasting*, *36*(3), 739–760.

Makridakis, Spyros, Spiliotis, Evangelos, & Assimakopoulos, Vassilios (2018). Statistical and machine learning forecasting methods: Concerns and ways forward. *PLoS One*, *13*(3).

Makridakis, Spyros, Spiliotis, Evangelos, & Assimakopoulos, Vassilios (2020). The M4 competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting*, *36*(1), 54–74.

Mariet, Zelda, & Kuznetsov, Vitaly (2019). Foundations of sequence-to-sequence modeling for time series. In *The 22nd international conference on artificial intelligence and statistics* (pp. 408–417).

McCracken, Michael W., & Ng, Serena (2016). FRED-MD: A monthly database for macroeconomic research. *Journal of Business & Economic Statistics*, *34*(4), 574–589.

McDonald, Daniel J., Shalizi, Cosma Rohilla, & Schervish, Mark (2017). Nonparametric risk bounds for time-series forecasting. *Journal of Machine Learning Research*, *18*(1), 1044–1083.

Montero-Manso, Pablo, Athanasopoulos, George, Hyndman, Rob J., & Talagala, Thiyanga S. (2020). FFORMA: Feature-based forecast model averaging. *International Journal of Forecasting*, *36*(1), 86–92.

NN5 forecasting competition for artificial neural networks and computational intelligence. (2018). http://www.neural-forecasting-competition.com/NN5/.

Oord, Aaron van den, Dieleman, Sander, Zen, Heiga, Simonyan, Karen, Vinyals, Oriol, Graves, Alex, et al. (2016). Wavenet: A generative model for raw audio. arXiv preprint arXiv:1609.03499.

Oreshkin, Boris N., Carpov, Dmitri, Chapados, Nicolas, & Bengio, Yoshua (2019). N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. In *International conference on learning representations*.

Pawlikowski, Maciej, & Chorowska, Agata (2020). Weighted ensemble of statistical models. *International Journal of Forecasting*, *36*(1), 93–97.

Rabanser, Stephan, Januschowski, Tim, Flunkert, Valentin, Salinas, David, & Gasthaus, Jan (2020). The effectiveness of discretization in forecasting: An empirical study on neural time series models. arXiv preprint arXiv:2005.10111.

Salinas, David, Flunkert, Valentin, Gasthaus, Jan, & Januschowski, Tim (2020). DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, *36*(3), 1181–1191.

Sen, Rajat, Yu, Hsiang-Fu, & Dhillon, Inderjit S. (2019). Think globally, act locally: A deep neural network approach to high-dimensional time series forecasting. In *Advances in neural information processing systems* (pp. 4837–4846).

Smyl, Slawek (2020). A hybrid method of exponential smoothing and recurrent neural networks for time series forecasting. *International Journal of Forecasting*, *36*(1), 75–85.

Sparks, Adam H., Carroll, Jonathan, Goldie, James, Marchiori, Dean, Melloy, Paul, Padgham, Mark, et al. (2020). bomrang: Australian Government Bureau of Meteorology (BOM) Data Client. R package version 0.7.0. https://CRAN.R-project.org/package=bomrang.

Taieb, Souhaib Ben, Bontempi, Gianluca, Atiya, Amir F., & Sorjamaa, Antti (2012). A review and comparison of strategies for multi-step ahead time series forecasting based on the NN5 forecasting competition. *Expert Systems with Applications*, *39*(8), 7067–7083.

Thung, Kim-Han, & Wee, Chong-Yaw (2018). A brief review on multi-task learning. *Multimedia Tools and Applications*, *77*(22), 29705–29725.

Trapero, Juan R., Kourentzes, Nikolaos, & Fildes, Robert (2015). On the identification of sales forecasting models in the presence of promotions. *Journal of the Operational Research Society*, *66*(2), 299–307.

Trinh, Trieu H., Dai, Andrew M., Luong, Minh-Thang, & Le, Quoc V. (2018). Learning longer-term dependencies in RNNs with auxiliary losses. arXiv preprint arXiv:1803.00144.

Štěpnička, Martin, & Burda, Michal (2017). On the results and observations of the time series forecasting competition CIF 2016. In *2017 IEEE international conference on fuzzy systems (FUZZ-IEEE)* (pp. 1–6).

Wang, Yuyang, Smola, Alex, Maddix, Danielle C., Gasthaus, Jan, Foster, Dean, & Januschowski, Tim (2019). Deep factors for forecasting. arXiv preprint arXiv:1905.12417.

Web traffic time series forecastin. (2017). https://www.kaggle.com/c/web-traffic-time-series-forecasting.

Zellner, Arnold (1962). An efficient method of estimating seemingly unrelated regressions and tests for aggregation bias. *Journal of the American Statistical Association*, *57*(298), 348–368.

Zhang, Yu (2015). Multi-task learning and algorithmic stability. In *Proceedings of the twenty-ninth AAAI conference on artificial intelligence* (pp. 3181–3187).